Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

React `useMemo` Hook

‹ Previous Next ›

The React `useMemo` Hook returns a memoized value.

Think of memoization as caching a value so that it does not need to be recalculated.

The `useMemo` Hook only runs when one of its dependencies update.

This can improve performance.

The `useMemo` and `useCallback` Hooks are similar:

`useMemo` returns a memoized value.

`useCallback` returns a memoized function.

Learn more about `useCallback` in the useCallback chapter.

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

Without useMemo

The `useMemo` Hook can be used to keep expensive, resource intensive functions from needlessly running.

In this example, we have an expensive function that runs on every render.

When changing the count or adding a todo, you will notice a delay in execution.

Example:

Get your own React.js Server

A poor performing function. The `expensiveCalculation` function runs on every render:

```
import { useState } from 'react';
import { createRoot } from 'react-dom/client';

const App = () => {
  const [count, setCount] = useState(0);
  const [todos, setTodos] = useState([]);
  const calculation = expensiveCalculation(count);

  const increment = () => {
    setCount((c) => c + 1);
  };
  const addTodo = () => {
    setTodos((t) => [...t, "New Todo"]);
  };

  return (
    <div>
      <div>
        <h2>My Todos</h2>
        {todos.map((todo, index) => {
          return <p key={index}>{todo}</p>;
        })}
        <button onClick={addTodo}>Add Todo</button>
```

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

```
        <button onClick={increment}>+</button>
        <h2>Expensive Calculation</h2>
        {calculation}
        <p>Note that this example executes the expensive function also wher
      </div>
    </div>
  );
};

const expensiveCalculation = (num) => {
  console.log("Calculating...");
  for (let i = 0; i < 1000000000; i++) {
    num += 1;
  }
  return num;
};

createRoot(document.getElementById('root')).render(
  <App />
);
```

Run Example »

W3Schools Tutorials Exercises
W3Schools app NEW iOS & Android
Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

Use `useMemo`

To fix this performance issue, we can use the `useMemo` Hook to memoize the `expensiveCalculation` function. This will cause the function to only run when needed.

We can wrap the expensive function call with `useMemo`.

The `useMemo` Hook accepts a second parameter to declare dependencies. The expensive function will only run when its dependencies have changed.

In the following example, the expensive function will only run when `count` is changed and not when todo's are added.

Example:

Performance example using the `useMemo` Hook:

```
import { useState, useMemo } from 'react';
import { createRoot } from 'react-dom/client';

const App = () => {
  const [count, setCount] = useState(0);
  const [todos, setTodos] = useState([]);
  const calculation = useMemo(() => expensiveCalculation(count), [count]);

  const increment = () => {
    setCount((c) => c + 1);
  };
  const addTodo = () => {
    setTodos((t) => [...t, "New Todo"]);
  };

  return (
```

W3Schools Tutorials Exercises
W3Schools app NEW iOS & Android
Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

```
            return <p key={index}>{todo}</p>;
          })}
          <button onClick={addTodo}>Add Todo</button>
        </div>
        <hr />
        <div>
          Count: {count}
          <button onClick={increment}>+</button>
          <h2>Expensive Calculation</h2>
          {calculation}
        </div>
      </div>
    );
};

const expensiveCalculation = (num) => {
  console.log("Calculating...");
  for (let i = 0; i < 1000000000; i++) {
    num += 1;
  }
  return num;
};

createRoot(document.getElementById('root')).render(
  <App />
);
```

Run Example »

Exercise ?

What is the main purpose of useMemo?

To memoize values to prevent unnecessary recalculations

To handle side effects

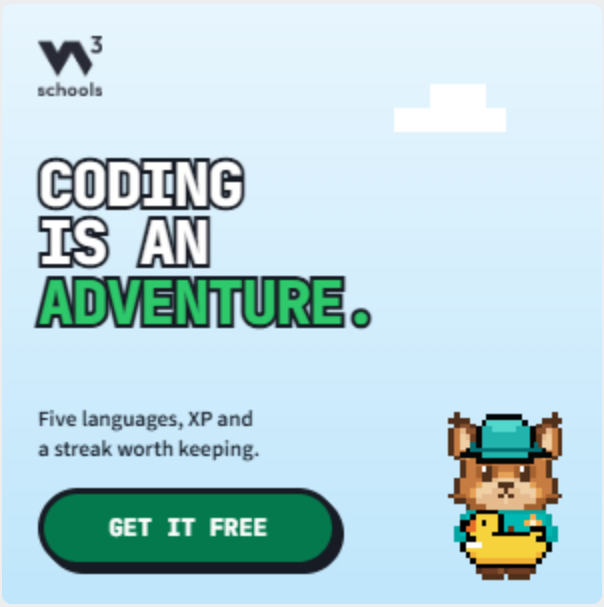
To manage component lifecycle

Submit Answer »

‹ Previous

Sign in to track progress

Next ›

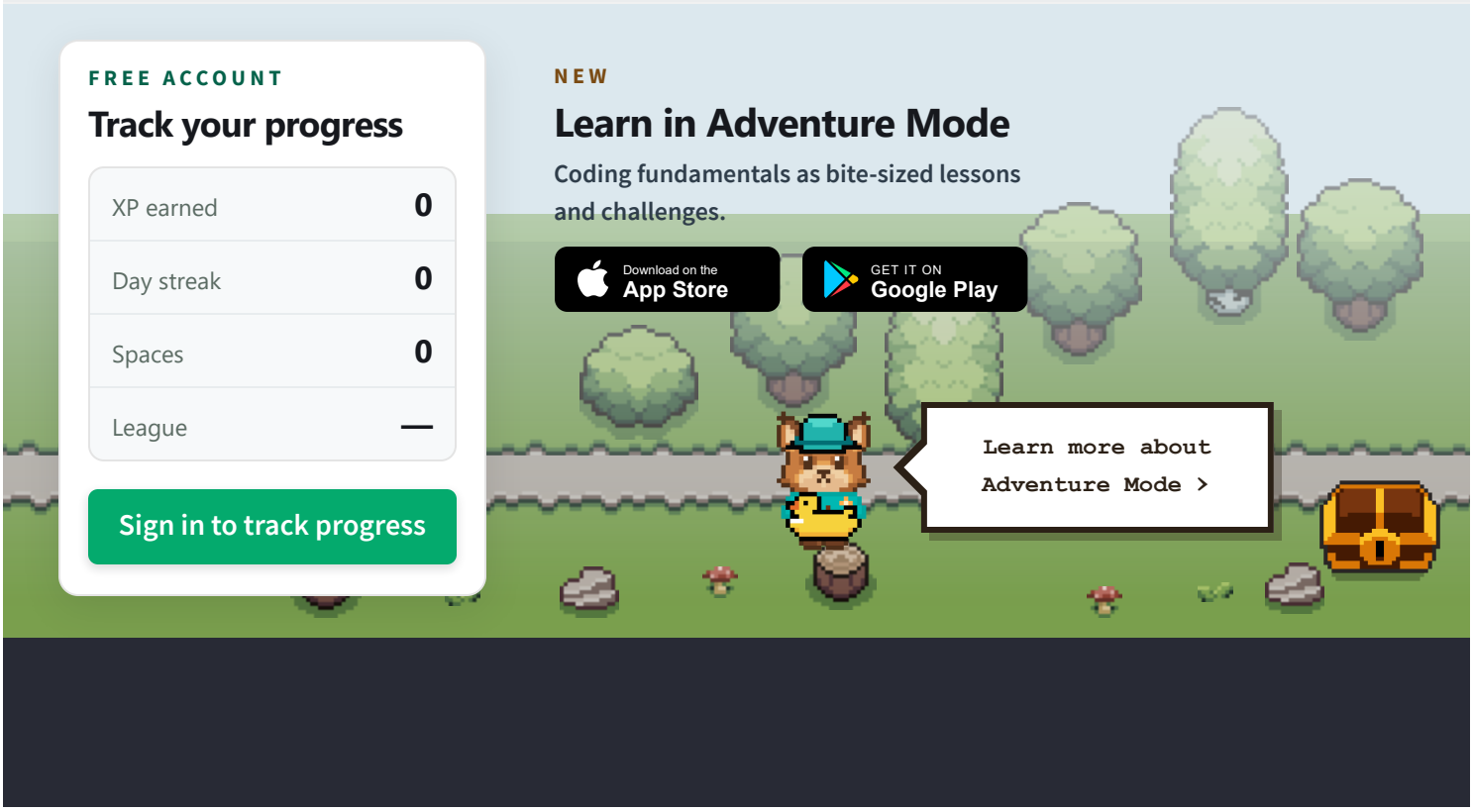


REMOVE ADS

Tutorials Exercises
W3Schools app iOS & Android NEW
Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

FREE ACCOUNT
Track your progress
XP earned 0
Day streak 0
Spaces 0
League —
Sign in to track progress

NEW
Learn in Adventure Mode
Coding fundamentals as bite-sized lessons and challenges.
Download on the App Store
GET IT ON Google Play
Learn more about Adventure Mode >



Tutorials Exercises

W3Schools app NEW
iOS & Android

Sign In

. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

GET CERTIFIED FOR TEACHERS

PRACTICE CONTACT US

Top Tutorials

HTML Tutorial
CSS Tutorial
JavaScript Tutorial
How To Tutorial
SQL Tutorial
Python Tutorial
W3.CSS Tutorial
Bootstrap Tutorial
PHP Tutorial
Java Tutorial
C++ Tutorial
jQuery Tutorial

Top References

HTML Reference
CSS Reference
JavaScript Reference
SQL Reference
Python Reference
W3.CSS Reference
Bootstrap Reference
PHP Reference
HTML Colors
Java Reference
AngularJS Reference
jQuery Reference

Top Examples

HTML Examples
CSS Examples
JavaScript Examples
How To Examples
SQL Examples
Python Examples
W3.CSS Examples
Bootstrap Examples
PHP Examples
Java Examples

Get Certified

HTML Certificate
CSS Certificate
JavaScript Certificate
Front End Certificate
SQL Certificate
Python Certificate
PHP Certificate
jQuery Certificate
Java Certificate
C++ Certificate

Tutorials Exercises

W3Schools app NEW
iOS & Android

Sign In

CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

FORUM ABOUT CAMPUS

W3Schools is optimized for learning and training. Examples might be simplified to improve reading and learning.
Tutorials, references, and examples are constantly reviewed to avoid errors, but we cannot warrant full correctness
of all content. While using W3Schools, you agree to have read and accepted our terms of use, cookies and privacy policy.

Copyright 1999-2026 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.